



UNIVALI

Universidade do Vale do Itajaí  
Escola do Mar, Ciência e Tecnologia - EMCT  
Ciência da Computação

---

# **PERT/CPM: CAMINHO CRÍTICO E FOLGAS**

---

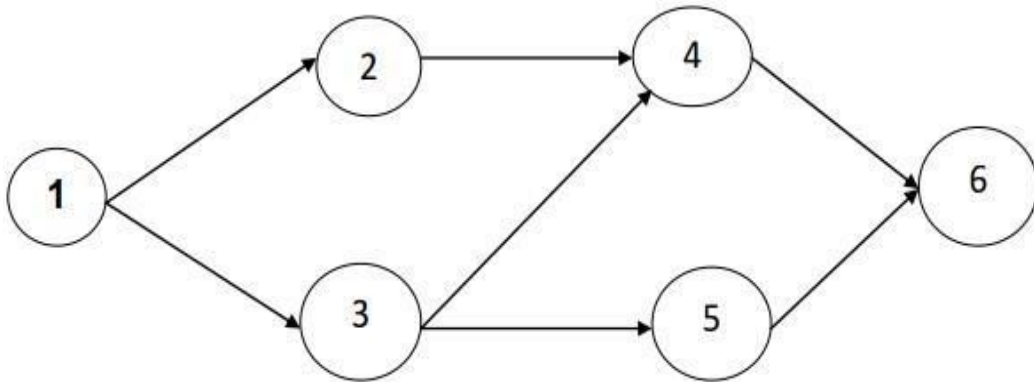
Gustavo Henrique Stahl Müller  
Paulo Henrique Rohling

---



# Problema

- Gerenciar um **conjunto de atividades** que devem ser executadas em uma **sequência específica**, onde mais de uma atividade pode ser realizada **ao mesmo tempo**;
- Definir atividades que são **mais críticas**, onde o descumprimento do prazo **atrasaria** o projeto como um todo.



| ATIVIDADE | DURAÇÃO |
|-----------|---------|
| 1         | 10 dias |
| 2         | 4 dias  |
| 3         | 7 dias  |
| 4         | 5 dias  |
| 5         | 5 dias  |
| 6         | 2 dias  |

# Solução

```
algoritmo CaminhoCrítico(grafo)
    EncontrarValoresDeInício(grafo)
    DeterminarTarefasFinais(grafo)
    EncontrarValoresDeTérmino(grafo)
fim-algoritmo
```

# Solução

**algoritmo** EncontrarValoresDeInício(grafo)  
    pilha += *EncontrarAdjacentes*(grafo.início)

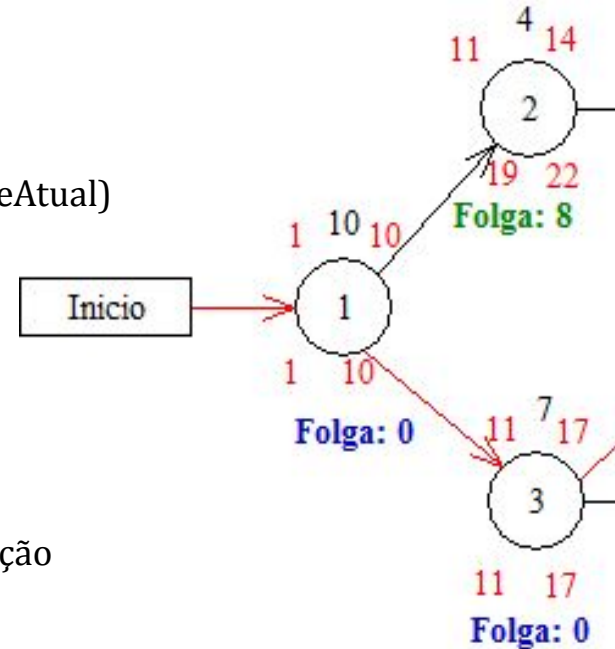
**enquanto** *NãoEstáVazia*(pilha) **faça**  
    vérticeAtual = *TopoDaPilha*(pilha)  
    precedente = *EncontrarPrecedenteComMaiorEarlyFinish*(vérticeAtual)  
    tempoDecorrido = precedente.earlyFinish

**se** *NãoEstáDefinido*(tempoDecorrido) **então**  
    tempoDecorrido = 0  
**fim-se**

    vérticeAtual.earlyStart = tempoDecorrido  
    vérticeAtual.earlyFinish = tempoDecorrido + vérticeAtual.duração  
    pilha += *EncontrarAdjacentes*(vérticeAtual)

**fim-enquanto**

**fim-algoritmo**



```
últimasAtividades = EncontrarVérticesSemAdjacentes(grafo)
maiorEarlyFinish = EncontrarMaiorEarlyFinish(últimasAtividades)
```

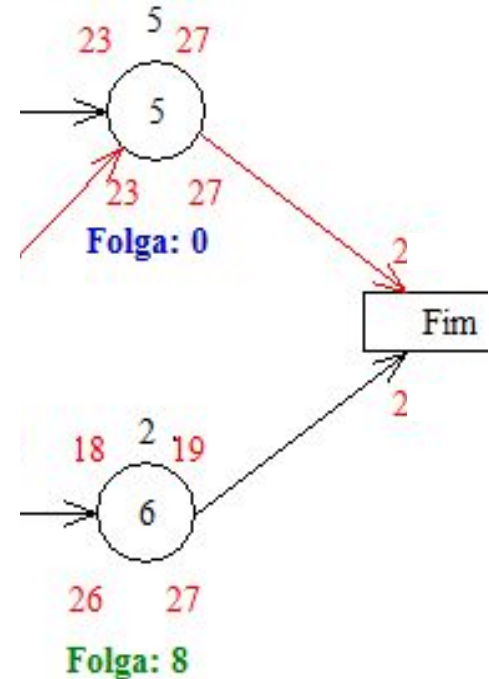
```
vértice.lateFinish = maiorEarlyFinish
```

$$\text{vértice.lateStart} = \text{maiorEarlyFinish} - \text{vértice.duração}$$
$$\text{vértice.slackTime} = \text{vértice.lateFinish} - \text{vértice.earlyFinish}$$

    pilha += *EncontrarPrecedentes*(vértice)

## fim-enquanto

# fim-algoritmo



# Solução

**algoritmo** EncontrarValoresDeTérmino(grafo)

**enquanto** *NãoEstáVazia*(pilha) **faça**

    vérticeAtual = *TopoDaPilha*(pilha)

    adjacente = *EncontrarAdjacenteComMenorLateStart*(vérticeAtual)

    tempoDeInícioDaTarefaSeguinte = adjacente.lateStart

    vérticeAtual.lateFinish = tempoDeInícioDaTarefaSeguinte

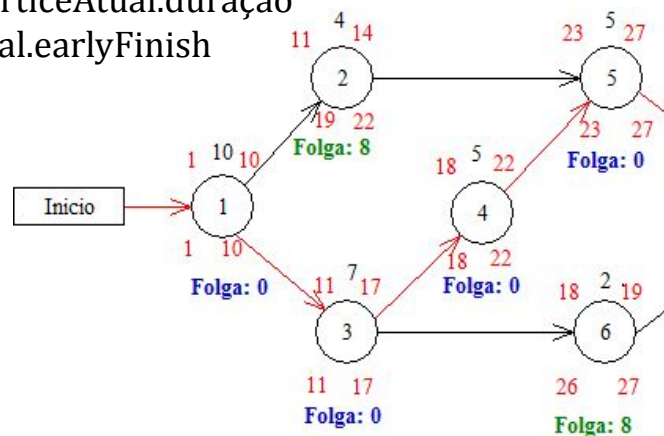
    vérticeAtual.lateStart = tempoDeInícioDaTarefaSeguinte - vérticeAtual.duração

    vérticeAtual.slackTime = vérticeAtual.lateFinish - vérticeAtual.earlyFinish

    pilha += *EncontrarPrecedentes*(vérticeAtual)

**fim-enquanto**

**fim-algoritmo**



# Solução

**algoritmo** PercorrerCaminhoCrítico(grafo)

vérticeAtual = grafo.início

**enquanto** *Verdadeiro* **faça**

adjacentesDoVérticeAtual = *EncontrarAdjacentes*(vérticeAtual)

próximaAtividadeCrítica = *EncontrarAtividadeComFolgaZero*(adjacentesDoVérticeAtual)

**se** *NãoEstáDefinido*(próximaAtividadeCrítica) **então**

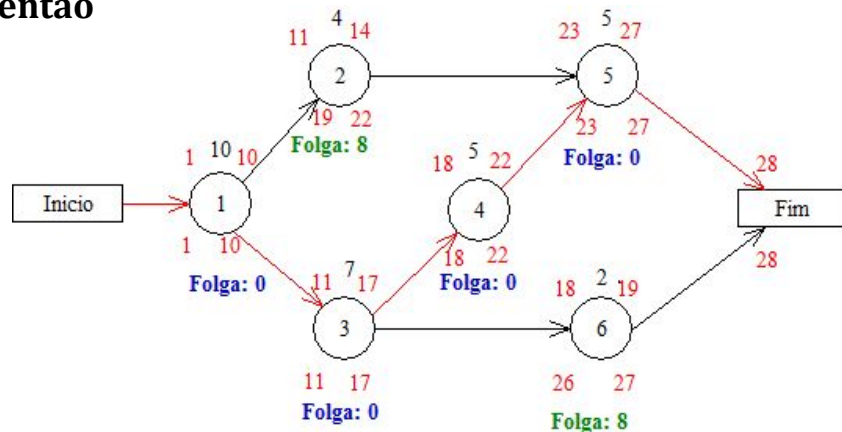
**fim-algoritmo**

**fim-se**

vérticeAtual = próximaAtividadeCrítica

**fim-enquanto**

**fim-algoritmo**



# Solução

**algoritmo** PercorrerCaminhoCrítico(grafo)  
    vérticeAtual = grafo.início

**enquanto** *Verdadeiro* **faça**

    adjacentesDoVérticeAtual = *EncontrarAdjacentes*(vérticeAtual)

    próximaAtividadeCrítica = *EncontrarAtividadeComFolgaZero*(adjacentesDoVérticeAtual)

**se** *NãoEstáDefinido*(próximaAtividadeCrítica) **então**

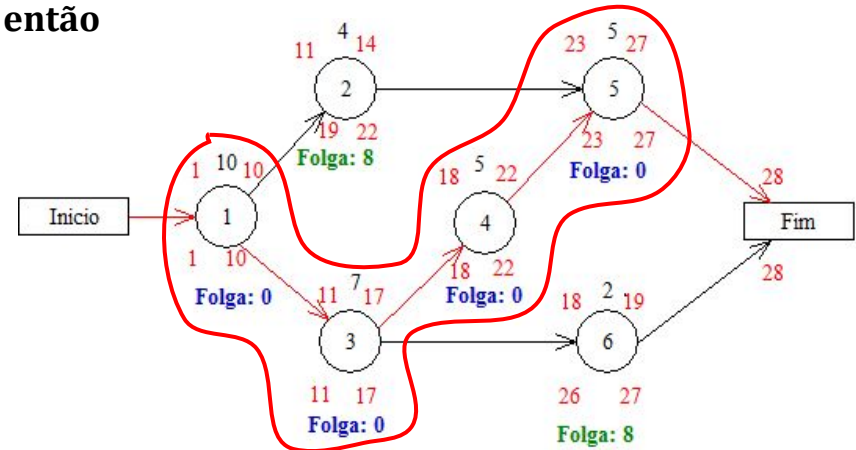
**fim-algoritmo**

**fim-se**

    vérticeAtual = próximaAtividadeCrítica

**fim-enquanto**

**fim-algoritmo**



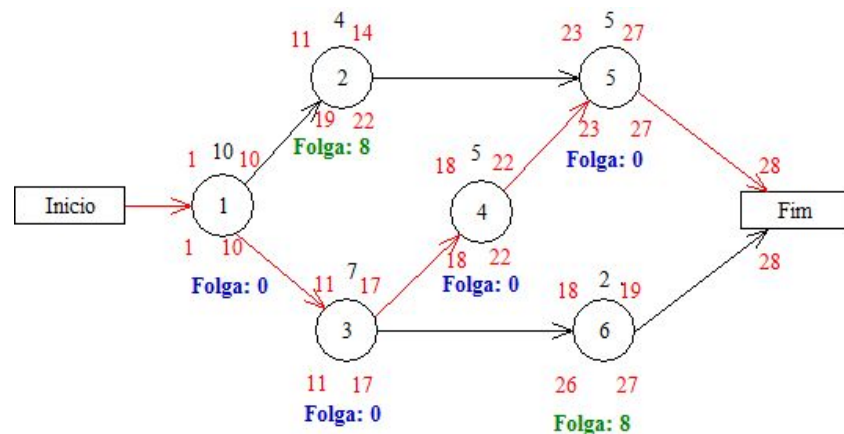


# Execução

```
#####  
                        CRITICAL PATH  
#####
```

| Activity | Duration | Precedents |
|----------|----------|------------|
| A        | 10       | [ ]        |
| B        | 4        | [A]        |
| C        | 7        | [A]        |
| D        | 5        | [C]        |
| E        | 5        | [B, D]     |
| F        | 2        | [C]        |

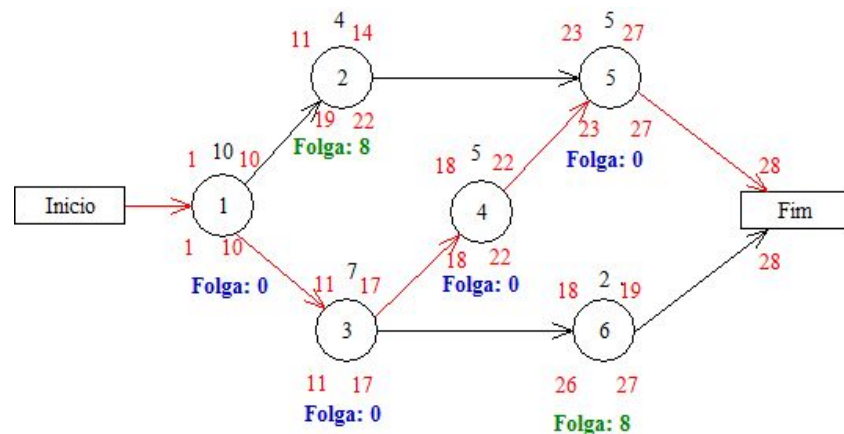
# Execução



| Activity | Early   | Late    | Slack time | Precedents |
|----------|---------|---------|------------|------------|
| A (10)   | 0   10  | 0   10  | 0          | []         |
| B (4)    | 10   14 | 18   22 | 8          | [A]        |
| C (7)    | 10   17 | 10   17 | 0          | [A]        |
| D (5)    | 17   22 | 17   22 | 0          | [C]        |
| E (5)    | 22   27 | 22   27 | 0          | [B, D]     |
| F (2)    | 17   19 | 25   27 | 8          | [C]        |

Critical path: @ -> A -> C -> D -> E -> END

# Execução



| Activity | Early   | Late    | Slack time | Precedents |
|----------|---------|---------|------------|------------|
| A (10)   | 1   10  | 1   10  | 0          | [ ]        |
| B (4)    | 11   14 | 19   22 | 8          | [A]        |
| C (7)    | 11   17 | 11   17 | 0          | [A]        |
| D (5)    | 18   22 | 18   22 | 0          | [C]        |
| E (5)    | 23   27 | 23   27 | 0          | [B, D]     |
| F (2)    | 18   19 | 26   27 | 8          | [C]        |

Critical path: @ -> A -> C -> D -> E -> END



UNIVALI

Universidade do Vale do Itajaí  
Escola do Mar, Ciência e Tecnologia - EMCT  
Ciência da Computação

---

# **PERT/CPM: CAMINHO CRÍTICO E FOLGAS**

---

Gustavo Henrique Stahl Müller  
Paulo Henrique Rohling

---

